

Tutorial 4 and 5

Marks: 30 | Due Date: 04/09/2026

What We're Doing Today

1. Adding a Business Tier to your application from last week
2. Adding a search function, and making it asynchronous with delegates
3. Doing the same thing again with async, await and Task
4. Adding an access log to the Business tier
5. Exception handling and awkward input

This worksheet covers two weeks. Tasks 1 and 2 are the Week 4 work; Tasks 3, 4 and 5 are the Week 5 work. Everything is submitted together at the due date above, but if you are not through Task 2 by the end of Week 4 you are behind — talk to your tutor.

Introduction

Well, after last week you should feel like you have a hang of this windows programming thing. Let's take things up a notch and do some more fun things with your network application. You're going to need to work a little harder here and figure things out for yourself, so I suggest you take a look at the code from this week's lecture for guidance... and ask your tutor if you get stuck!

Task 1: Adding a Business Tier [5 marks]

Before you start: make your Data tier a singleton if it is not!

WCF creates a new `DataService` object for every connection it receives. If your database is built in that object's constructor, then every new connection builds a brand new database full of different randomly generated people. Fix it before you go any further. Please take a look at tutorial 2-3 solutions if required.

The database needs to be created once and shared by every connection. The usual way is a singleton:

```
public class Database
{
    private readonly List<DataStruct> _database;

    public static Database Instance { get; } = new Database();
    static Database() { }

    private Database()
    {
        // build the records here, once
    }
    // ... your GetXByIndex functions
}
```

Note the private constructor — it stops anything else creating a second database. Your `DataService` then just holds a reference to it:

```
private readonly Database _db = Database.Instance;
```

Because every connection now shares one object, that object is being read by many threads at once. That is fine here, because nothing writes to it after construction. Keep it that way, and keep `ConcurrencyMode.Multiple` on your service behaviour.

Building the tier

First things first, you need to add a Business Tier. This means you'll need to create a new Console project inside your solution. This console program needs to sit between your GUI and your Data tier project.

Your business tier is going to look a little like your Data tier, presenting an interface and an implementation to your GUI... however it's also going to have code reminiscent of your GUI in the implementation of those functions, as they are going to call the functions in the Data tier.

You will want to start by creating a new public interface called `BusinessServerInterface` that replicates all the functionality of the Data tier. Remember to tag it `[ServiceContract]` and each function `[OperationContract]`, as you did last week.

Then, you'll want to create an implementation called `BusinessServer` that calls the required functions on the Data tier through a `ChannelFactory<DataServerInterface>`.

Your Business tier needs its own port. The Data tier is on 8100, so give the Business tier something else — 8200 works. Host it the same way you hosted the Data tier last week.

After you've written this, change the reference on your GUI to point to your Business tier rather than your Data tier, and change the `ChannelFactory<DataServerInterface>` to a `ChannelFactory<BusinessServerInterface>` accordingly.

If you've done it right, spinning up the Data tier, then the Business tier, and then the GUI will allow the program to run nicely. If you don't start the Business tier, the GUI should consequentially not be able to work. Once you've got the Business tier between the GUI and the Data tier, you can move on.

Task 2: Search [10 marks]

You're going to add a search capability to your Business tier!

Setting up the Business Tier for search

The business tier needs to implement a function that will take in a string and return the contents of the first record that has a last name matching the string.

First things first, we're going to make this a hard thing to do — make sure your Data tier generates at least 100,000 records in its database.

Next, implement the function. Remember to include it in the Business tier interface so that the GUI can actually use it! Decide what your function returns, and write it down before you start coding — you need to know what a match looks like, and what a failed search looks like, before your GUI can do anything sensible with the answer.

Once you've got that all done, move on to...

Adding to the GUI so you can search

You will need to (at the very least) add a text box and button that will allow you to search the database.

The on-click function of the new button you add should take the string, call the search method of the Business tier via RPC, and then set the values of the other text boxes to the values returned by the function.

Test with a name that isn't there. Your search stops at the first match, so a common surname is found almost immediately and everything feels fast. A surname that does not exist forces a scan of all 100,000 records, and that is where you will see the delay — and watch your GUI lock up completely while it happens. Try both before you decide whether you have a problem.

This freeze is the perfect kind of function to use an asynchronous call for!

Reading material

<https://www.c-sharpcorner.com/uploadfile/vendettamit/delegates-and-async-programming/>

<https://www.c-sharpcorner.com/UploadFile/vendettamit/delegate-and-async-programming-C-Sharp-asynccallback-and-object-state/>

<https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/concepts/async/>

Making it asynchronous

You will want to create a delegate.

A delegate is basically a specialised function pointer used by C#. So long as it is of a delegate type with the same function parameters as a function you want to call, it will allow you to call it without actually calling the function.

An example, for a calculator class as follows:

```
namespace MyExample {
    public class Calculator
    {
        public int Add(int num1, int num2) { return num1 + num2; }
        public int Subtract(int num1, int num2) { return num1 - num2; }
        public int Multiply(int num1, int num2) { return num1 * num2; }
    }
}
```

You can create and use a Delegate as follows:

```
// delegate definition
public delegate int BinaryOperation(int operand1, int operand2);

public class DelegateTester
{
    public static void Main() {
        BinaryOperation binOp;           // a variable of delegate type
        Calculator calc;

        calc = new Calculator();
        binOp = calc.Add;                 // point at Add() of calc (the object!)
        System.Console.WriteLine("1 + 2 = " + binOp(1, 2));
        binOp = calc.Subtract;           // point at calc.Subtract()
        System.Console.WriteLine("1 - 2 = " + binOp(1, 2));
    }
}
```

You can also Invoke a delegate to make the function execute in a separate thread:

```
public delegate int BinaryOperation(int operand1, int operand2);

public class CalcClient
{
    public static void Main() {
        BinaryOperation addDel;
        Calculator calc = new Calculator();
        IAsyncResult asyncObj1, asyncObj2; // each tracks one async call
        int iAddResult1, iAddResult2;

        addDel = calc.Add;                // point delegate at the function

        // start worker threads
        asyncObj1 = addDel.BeginInvoke(1, 2, null, null);
        asyncObj2 = addDel.BeginInvoke(3, 4, null, null);

        // Block, waiting for all async calls to complete
    }
}
```

```

        iAddResult1 = addDel.EndInvoke(asyncObj1);
        iAddResult2 = addDel.EndInvoke(asyncObj2);

        asyncObj1.AsyncWaitHandle.Close();    // clean up
        asyncObj2.AsyncWaitHandle.Close();

        System.Console.WriteLine("1 + 2 = " + iAddResult1);
        System.Console.WriteLine("3 + 4 = " + iAddResult2);
    }
}

```

And you can even get those threads to perform callback functions, which execute asynchronously once the delegate finishes.

```

public delegate int BinaryOperation(int operand1, int operand2);

public class CalcClient
{
    public static void Main() {
        BinaryOperation addDel;
        AsyncCallback callbackDel;    // .NET type used by BeginInvoke()
        Calculator calc = new Calculator();

        addDel = calc.Add;
        callbackDel = this.OnAddCompletion;    // point at callback function
        addDel.BeginInvoke(1, 2, callbackDel, null);

        System.Console.WriteLine("Waiting for completion.");
        System.Console.ReadLine();
    }

    // must match the AsyncCallback signature
    private void OnAddCompletion(IAsyncResult asyncResult) {
        int iAddResult;
        BinaryOperation addDel;
        AsyncResult asyncObj = (AsyncResult)asyncResult;    // for AsyncDelegate

        if (asyncObj.EndInvokeCalled == false) {    // never EndInvoke twice!
            addDel = (BinaryOperation)asyncObj.AsyncDelegate;
            // ref / out params come back here too
            iAddResult = addDel.EndInvoke(asyncObj);
            System.Console.WriteLine("1 + 2 = " + iAddResult);
        }
        asyncObj.AsyncWaitHandle.Close();
    }
}

```

The cast to AsyncResult needs using `System.Runtime.Remoting.Messaging`; at the top of your file.

The waiting state

This last one is the most important. What you can do is put the GUI into a “waiting state”, which is still responsive, but doesn’t let the user do anything... and then wake it up using the completion callback.

You should add a progress bar to your GUI. When your search button is clicked, set your text boxes’ `IsReadOnly` to true, disable the buttons on your GUI, and set your `ProgressBar`’s `IsIndeterminate` to true — this gives you the animated “something is happening, duration unknown” bar. Then call the RPC function for search on the Business tier.

Getting back to the UI thread

Now, an important catch. Your completion callback does *not* run on the GUI thread — it runs on the worker thread that carried out the search. WPF controls may only be touched by the thread that created them, so

writing to your text boxes directly from the callback will throw an exception. You need to hand the work back to the GUI thread using the control's Dispatcher, like this:

```
SName.Dispatcher.Invoke(new Action(() => SName.Text = aStudent.Name));
```

So in the completion callback, use the Dispatcher to set all the values of your text boxes, set `IsReadOnly` back to false, enable all buttons, and set the `ProgressBar`'s `IsIndeterminate` back to false. Every one of those is a GUI operation, so every one of them needs to go through the Dispatcher.

What you get back

Your search returns nothing meaningful when there is no matching last name. Decide what that looks like — a null, an empty record, a flag — and make sure your callback checks for it before it tries to read fields off the result. Otherwise, your program will crash on the first failed search, which, as noted above, is exactly the search you should be testing with. You'll be asked about this again in Task 5.

Congratulations! You've made a responsive application using asynchronous delegates. I have added a solution "delegateExample.zip" in Lecture 4 as well. It will come in handy — note that it demonstrates the delegate and Dispatcher mechanics only. It does not implement the waiting state described above, and it does not handle the no-match case. Both of those are yours to write.

Task 3: Making the same functionality using Async, Await and Task [5 marks]

Create a new client app using WPF. This program will do the same as your previous client. However, this time you will use `Async`, `Await` and `Task`. This is another way of making a program asynchronous. I have added a solution `AsynHandle.zip` for your reference in Lecture 4. Note: this is a simple and cleaner way to handle threading in other languages as well, e.g., Javascript's 'promise'.

Compare the two clients carefully — this is the point of the exercise. Open `UpdateGui` in both solutions. In the delegate version, every line goes through the Dispatcher. In the `async/await` version, the same lines assign directly to the text boxes with no Dispatcher at all. That is not a stylistic difference. When execution resumes after an `await`, it resumes on the thread that started it — the GUI thread — so no marshalling is needed. The delegate version hands you a worker thread and leaves the thread affinity problem for you to solve; `async/await` solves it for you.

Your `async` client needs the same waiting state and the same no-match handling as the delegate client. The reference solution shows the mechanism, not a complete answer.

Task 4: Adding an access log [5 marks]

The next thing to do is to add an access log to the Business tier so we know what is happening. This is in general a very good thing to do with Business tier applications, as the logic generally has business implications, and executives like to have something to look at to know that their applications are actually doing the right thing.

What we're going to do is build a `Log(string logString)` function, and call it from every function that can be called on the Business tier. It will use a counter that keeps track of how many log-able tasks have been performed. Both the counter and the function need to be **static**:

```
private static uint LogNumber = 0;

private static void Log(string logString) { ... }
```

This function should not be in the interface, as you do not want clients calling it directly. It's just for the Business tier itself.

Your function needs to write out how many tasks have been performed thus far, and the log string that was sent to it. This string needs to be informative. As informative as possible. Put as much information as you can into it — which function was called, what arguments came in, when, and what came back. Write it to the Business tier's console so you can watch it live, and append it to a text file so there's something left over afterwards.

As you can probably guess, there could be issues if multiple clients try to run this function at the same time, and it's likely going to happen that way as every function in the Business tier will call the log function. Two clients can both read `LogNumber` as 7, both write 8, and you've silently lost a count. Worse, two threads writing to the same log file at the same moment will throw. So we need to do some automatic thread synchronisation.

Thankfully, .NET has our back with an auto-thread-sync tool. Tag your Log function with the attribute `[MethodImpl(MethodImplOptions.Synchronized)]` (you'll need using `System.Runtime.CompilerServices`). This tells .NET to only allow one thread at a time to run this function.

Why does it have to be static?

When your Business tier runs, WCF does not create one `BusinessServer` object — it creates a fresh one for each client that connects. `MethodImplOptions.Synchronized` on a normal (instance) method locks the object the call arrived on, so each client would end up locking its *own* object and none of them would ever wait for each other. The counter and the log file, meanwhile, are shared by all of them. That is the race condition above, and the attribute would do nothing to stop it.

Marking the method static changes what gets locked: .NET locks the class itself, and there is only ever one of those. So, one lock now guards one shared counter and one shared file, and the synchronisation actually does its job.

```
[MethodImpl(MethodImplOptions.Synchronized)]
private static void Log(string logString)
{
    LogNumber++;
    var line = $"{LogNumber}: {DateTime.Now:G} - {logString}";
    Console.WriteLine(line);
    File.AppendAllText("BusinessLog.txt", line + Environment.NewLine);
}
```

Make sure `LogNumber++` happens **inside** the synchronised function. If you increment it in the calling function and pass the number in, you're back to an unprotected read-modify-write and the lock buys you nothing.

Prove it works. Start two copies of your GUI and have both of them search at the same time. Your log should be one continuous, correctly numbered sequence covering both clients — not two interleaved sequences, and not a file-in-use exception.

Task 5: Cleaning up [5 marks]

As before, make sure you have good exception handling and can deal with unusual input. What if, for example:

6. Someone searches a name with no match?
7. Someone tries to search a number? Or a special character?
8. Something goes wrong during the search?

Remember that you now have *two* network boundaries, not one. An exception thrown in the Data tier has to get through the Business tier and out to the GUI before anyone can see it, and ordinary exceptions do not cross a WCF boundary. You solved this once already for the Data tier — look back at your fault contracts and do the same for the Business tier.

You should also implement a timeout, so a search that never comes back doesn't leave your GUI stuck in its waiting state forever. The tidy way is to set `SendTimeout` on your `NetTcpBinding` and handle the resulting `TimeoutException`. Be careful with approaches that block waiting for the result — that will re-freeze the GUI you just spent Task 2 unfreezing.

Submission

Zip your entire solution directory — leave nothing out — and submit it to the tutorial 4-5 section on Blackboard before the due date. **Re-download and open your submission to check it went up correctly.**

You will demonstrate your work to your tutor and answer questions about it. Be ready to explain why your log function is static, why the callback needs the Dispatcher, and what happens when two clients search at once.